

// Jivops

Calico

Guía completa — 30 lecciones

Fundamentos · NetworkPolicy · Encapsulación y networking · eBPF ·
Seguridad avanzada · Producción avanzada

Contenido

Módulo 1: Fundamentos de Calico

- Lección 01 — Qué es Calico y su enfoque distintivo frente a otros CNI
- Lección 02 — Arquitectura de Calico — Felix, BIRD, Typha, confd
- Lección 03 — Enrutamiento nativo con BGP — sin overlay por defecto
- Lección 04 — IPAM de Calico — asignación de direcciones IP
- Lección 05 — Instalación de Calico en un cluster de Kubernetes

Módulo 2: NetworkPolicy en profundidad

- Lección 06 — Qué es NetworkPolicy y el modelo deny-by-default
- Lección 07 — Selectors — namespaceSelector y podSelector
- Lección 08 — Reglas de ingress y egress
- Lección 09 — GlobalNetworkPolicy — políticas a nivel de cluster
- Lección 10 — Orden de evaluación y prioridad de políticas

Módulo 3: Modos de encapsulación y networking

- Lección 11 — Modo BGP puro vs IP-in-IP
- Lección 12 — VXLAN en Calico — cuándo se necesita
- Lección 13 — Cross-subnet networking
- Lección 14 — MTU en Calico
- Lección 15 — Calico con topologías de red complejas (on-prem, cloud)

Módulo 4: eBPF dataplane

- Lección 16 — Qué es eBPF y por qué Calico lo adoptó como dataplane alternativo
- Lección 17 — eBPF vs iptables — diferencias de rendimiento
- Lección 18 — DSR (Direct Server Return) con eBPF
- Lección 19 — Habilitar y migrar hacia el dataplane eBPF
- Lección 20 — Limitaciones del dataplane eBPF

Módulo 5: Seguridad avanzada

- Lección 21 — Calico Enterprise/Calico Cloud vs Calico OSS
- Lección 22 — Application layer policy con Istio
- Lección 23 — Cifrado de datos en tránsito con WireGuard
- Lección 24 — DNS policy — restringir acceso por dominio
- Lección 25 — Federación de clusters con Calico

Módulo 6: Producción avanzada

Lección 26 — Troubleshooting de Calico — calicoctl y diagnóstico

Lección 27 — Monitoreo de Calico con Prometheus

Lección 28 — Rendimiento y escalabilidad de Calico en clusters grandes

Lección 29 — Migración hacia Calico desde otro CNI

Lección 30 — Proyecto final: red de cluster con Calico lista para producción

Módulo 1: Fundamentos de Calico

Lección 01 — Qué es Calico y su enfoque distintivo frente a otros CNI

Calico prioriza el enrutamiento nativo de capa 3 usando el protocolo BGP para propagar rutas reales entre nodos, evitando la encapsulación de paquetes siempre que sea posible, además de ofrecer soporte nativo y avanzado de NetworkPolicy desde su diseño original.

Evitar la encapsulación elimina el trabajo adicional de procesar cada paquete individual añadiendo y luego quitando una capa extra de encabezado en cada extremo de la comunicación; el enrutamiento nativo de Calico usando BGP aprovecha rutas reales de la red física, un principio similar al tradeoff ya visto entre host-gw y VXLAN en el curso de Flannel, donde evitar la encapsulación mejora el rendimiento a cambio de ciertos requisitos específicos sobre la topología de red real disponible.

Lección 02 — Arquitectura de Calico — Felix, BIRD, Typha, confd

Felix es el agente principal que programa las reglas de red reales en el nodo (rutas, reglas de firewall para NetworkPolicy), mientras BIRD es el daemon de enrutamiento BGP que propaga y recibe información de rutas hacia y desde otros nodos del cluster.

Programar reglas de firewall local para aplicar NetworkPolicy es una tarea técnica distinta de gestionar el protocolo de comunicación BGP con otros nodos del cluster; al separar esas responsabilidades entre Felix y BIRD, cada componente puede enfocarse completamente en resolver bien su tarea específica, facilitando el mantenimiento, la depuración y la evolución independiente de cada pieza del sistema sin mezclar lógicas distintas dentro del mismo componente.

Lección 03 — Enrutamiento nativo con BGP — sin overlay por defecto

Los paquetes viajan directamente usando rutas reales de la red física, sin el overhead de encapsular y desencapsular cada paquete, ofreciendo mejor rendimiento y facilitando el diagnóstico de problemas de red al poder inspeccionar el tráfico real sin esa capa adicional de abstracción.

Con tráfico encapsulado dentro de una red overlay, una herramienta de diagnóstico como tcpdump capturaría primero el paquete encapsulado, requiriendo un paso adicional de desencapsulación manual antes de poder ver el contenido real del paquete original; con enrutamiento nativo BGP, el paquete capturado es directamente el paquete real sin esa capa adicional, facilitando un diagnóstico más directo y simple usando herramientas estándar de red.

Lección 04 — IPAM de Calico — asignación de direcciones IP

Gestiona la asignación eficiente de direcciones IP del rango total disponible hacia cada Pod nuevo, organizando ese rango en bloques más pequeños asignados a cada nodo, evitando conflictos de direcciones duplicadas entre Pods de distintos nodos del cluster.

Si cada nodo ya tiene asignado su propio bloque de direcciones, puede asignar direcciones a sus Pods locales de forma inmediata sin necesitar consultar ni coordinar con un pool central compartido en cada ocasión; sin esos bloques preasignados, cada nueva creación de Pod en cualquier nodo tendría que coordinar con ese recurso central compartido, lo que podría convertirse en un cuello de botella real con mucha actividad simultánea de creación de Pods en el cluster.

Lección 05 — Instalación de Calico en un cluster de Kubernetes

Aplicar los manifiestos YAML oficiales de Calico (que incluyen DaemonSets para Felix y BIRD en cada nodo, los recursos personalizados necesarios, y la configuración de IPAM), confirmando que el rango de direcciones de Pods configurado coincida con el usado al inicializar el cluster, de forma similar a como se vio con Flannel.

Kubernetes ya tiene definido un rango específico de direcciones de Pods desde el momento en que se inicializó el cluster, independientemente de cuál plugin CNI se vaya a instalar después; ese requisito de coincidencia entre lo que Kubernetes espera y lo que el plugin CNI configura aplica de la misma forma tanto para Calico como para Flannel, siendo una consideración fundamental sobre la configuración de rangos de direcciones que no depende de la elección específica de plugin realizada.

Módulo 2: NetworkPolicy en profundidad

Lección 06 — Qué es NetworkPolicy y el modelo deny-by-default

Una vez que al menos una NetworkPolicy selecciona a ese Pod, todo el tráfico no explícitamente permitido por alguna regla de esa política (o de otras políticas que también lo seleccionen) queda denegado por defecto, invirtiendo el comportamiento de conectividad completamente abierta que existía antes de aplicar cualquier política.

Bajo el modelo deny-by-default, una vez que la política aplica a ese Pod, todo el tráfico no explícitamente permitido por alguna regla queda automáticamente denegado; como la política solo tiene una regla que permite tráfico desde un origen específico, cualquier otro origen distinto que no esté mencionado en ninguna regla de permiso queda bloqueado automáticamente, sin necesitar una regla de denegación explícita adicional para ese otro origen.

Lección 07 — Selectors — namespaceSelector y podSelector

Un podSelector selecciona Pods específicos según sus etiquetas (labels), típicamente dentro del mismo namespace, mientras un namespaceSelector selecciona todos los Pods que pertenecen a namespaces que coincidan con ciertas etiquetas, permitiendo definir reglas que abarquen potencialmente múltiples namespaces distintos.

Si la necesidad real es permitir tráfico desde cualquier Pod que pertenezca al namespace de monitoreo, sin importar las etiquetas específicas de cada Pod individual, un namespaceSelector que seleccione ese namespace por su propia etiqueta abarcaría automáticamente a todos los Pods que vivan dentro de él; un podSelector, en cambio, requeriría conocer y especificar las etiquetas exactas de cada Pod individual, algo que no se ajusta bien a esa necesidad de incluir cualquier Pod del namespace completo.

Lección 08 — Reglas de ingress y egress

Las reglas de ingress controlan qué tráfico entrante está permitido hacia el Pod seleccionado (quién puede conectarse hacia él), mientras las reglas de egress controlan qué tráfico saliente está permitido desde ese Pod hacia otros destinos (hacia dónde puede conectarse ese Pod).

Ingress y egress son dimensiones independientes de control dentro de una NetworkPolicy, cada una regulando una dirección distinta del tráfico; si la política solo define reglas de ingress, la restricción deny-by-default se activa únicamente para esa dirección (quién puede entrar), mientras la dirección de egress permanece sin esa restricción activada, ya que no se definió ninguna regla que active específicamente el control sobre el tráfico saliente del Pod.

Lección 09 — GlobalNetworkPolicy — políticas a nivel de cluster

Una GlobalNetworkPolicy puede aplicarse a Pods a través de múltiples namespaces distintos simultáneamente (o incluso a nodos completos), útil para definir reglas de seguridad consistentes que deben aplicarse de forma centralizada a todo el cluster, sin tener que replicar la misma política individualmente en cada namespace.

Replicar manualmente esa misma regla de seguridad en cada namespace individual requeriría mantener múltiples copias idénticas distribuidas por todo el cluster, y actualizar esa regla en el futuro significaría modificar cada una de esas copias por separado; una GlobalNetworkPolicy centraliza esa regla en un único lugar, aplicándose automáticamente a todos los namespaces sin esa replicación manual repetitiva, garantizando consistencia y reduciendo significativamente el esfuerzo de mantenimiento.

Lección 10 — Orden de evaluación y prioridad de políticas

Calico evalúa las políticas aplicables según un orden determinado por su prioridad configurada (las GlobalNetworkPolicy suelen tener mecanismos explícitos de orden), y una regla de denegación explícita en una política de mayor prioridad puede anular el permiso otorgado por una política de menor prioridad, haciendo importante entender ese orden al diseñar políticas que interactúan entre sí.

Si ambas políticas tienen reglas potencialmente contradictorias (una permite cierto tráfico, la otra lo deniega), el comportamiento final real del Pod depende directamente de cuál de las dos políticas tiene mayor prioridad según el orden configurado; sin verificar cuidadosamente ese orden, no se podría predecir con certeza si el tráfico específico en cuestión terminará siendo permitido o denegado, pudiendo generar un comportamiento inesperado si la prioridad real no coincide con lo que se asumía al diseñar esas políticas.

Módulo 3: Modos de encapsulación y networking

Lección 11 — Modo BGP puro vs IP-in-IP

El modo BGP puro requiere que los nodos puedan intercambiar rutas directamente entre sí sin obstáculos (como estar en la misma red de capa 2 o tener BGP habilitado entre subredes), una condición que no siempre se cumple; IP-in-IP ofrece una alternativa que funciona en topologías de red donde ese enrutamiento directo no es viable.

Muchas infraestructuras de nube pública no permiten que un cluster propague libremente sus propias rutas BGP personalizadas a través de la red subyacente gestionada por el proveedor; sin esa capacidad, el modo BGP puro no podría funcionar correctamente porque los nodos no tendrían forma real de conocer las rutas directas entre sí, mientras IP-in-IP resuelve ese problema encapsulando el tráfico de una forma que no depende de esa propagación de rutas personalizadas en la infraestructura subyacente.

Lección 12 — VXLAN en Calico — cuándo se necesita

En entornos donde el tráfico IP-in-IP específicamente podría ser bloqueado por el firewall de la infraestructura subyacente (algunos proveedores cloud bloquean ese protocolo específico), VXLAN ofrece una alternativa de encapsulación que usa el puerto UDP estándar, generalmente más compatible con configuraciones de firewall restrictivas.

Si ese proveedor cloud bloquea específicamente el protocolo que usa IP-in-IP, ese backend simplemente no funcionaría dentro de esa infraestructura particular; VXLAN, al usar un puerto UDP estándar generalmente permitido por las mismas restricciones de firewall, se convierte en la alternativa de encapsulación viable para ese entorno específico, donde IP-in-IP no podría atravesar esa restricción particular impuesta por el proveedor.

Lección 13 — Cross-subnet networking

Permite que Calico use enrutamiento nativo sin encapsulación cuando dos nodos están en la misma subred (donde el tráfico directo funciona bien), pero recurra automáticamente a la encapsulación (IP-in-IP o VXLAN) específicamente cuando el tráfico necesita cruzar entre subredes distintas donde el enrutamiento directo no es viable.

La mayoría del tráfico real dentro de un cluster podría ocurrir entre nodos de la misma subred, donde el enrutamiento nativo sin encapsulación ofrece el mejor rendimiento posible; encapsular absolutamente todo el tráfico sin distinción desperdiciaría ese beneficio de rendimiento incluso en los casos donde no era realmente necesario, mientras aplicar encapsulación solo de forma selectiva para el tráfico que efectivamente cruza entre subredes distintas preserva el rendimiento óptimo para la mayoría del tráfico, usando encapsulación únicamente donde realmente se requiere.

Lección 14 — MTU en Calico

Cualquier forma de encapsulación añade una cabecera adicional a cada paquete, reduciendo el espacio real disponible para los datos originales; si el MTU no se ajusta considerando ese overhead, los paquetes pueden fragmentarse innecesariamente o descartarse, el mismo problema fundamental ya visto con VXLAN en Flannel.

El principio fundamental de que cualquier mecanismo de encapsulación añade una cabecera adicional que reduce el espacio disponible para los datos originales es el mismo sin importar cuál proyecto CNI específico esté aplicando esa técnica; tanto VXLAN en Flannel como IP-in-IP o VXLAN en Calico

encapsulan el tráfico de forma similar, así que el mismo riesgo real de fragmentación por exceder el MTU disponible aplica igualmente en ambos casos, siendo una consecuencia de usar encapsulación en general, no una particularidad exclusiva de un único proyecto CNI determinado.

Lección 15 — Calico con topologías de red complejas (on-prem, cloud)

Permite configurar el modo más apropiado según las características específicas de cada segmento de la infraestructura (por ejemplo, BGP puro donde el datacenter on-premise lo permite, y VXLAN donde la restricción de un proveedor cloud específico lo requiere), adaptándose a topologías de red heterogéneas dentro del mismo cluster o entre clusters federados.

Si se forzara un único modo uniforme para toda la infraestructura, el segmento on-premise sacrificaría innecesariamente el mejor rendimiento que BGP puro podría ofrecer ahí, mientras el segmento del proveedor cloud restrictivo necesitaría específicamente VXLAN debido a sus limitaciones particulares; la flexibilidad de Calico para configurar el modo apropiado según cada segmento permite optimizar cada parte de la infraestructura según sus características reales, sin sacrificar rendimiento en el segmento más favorable solo por mantener una configuración uniforme forzada en toda la infraestructura heterogénea.

Módulo 4: eBPF dataplane

Lección 16 — Qué es eBPF y por qué Calico lo adoptó como dataplane alternativo

eBPF es una tecnología del kernel de Linux que permite ejecutar programas personalizados y eficientes directamente dentro del kernel sin modificar su código fuente, y Calico lo adoptó porque permite procesar el tráfico de red de forma más eficiente que las cadenas de reglas de iptables, especialmente notable en clusters grandes con muchas reglas.

Iptables evalúa las reglas de una cadena de forma secuencial, una por una, hasta encontrar la que realmente aplica al paquete que se está procesando; a medida que el número de reglas acumuladas crece significativamente (con muchas NetworkPolicy en un cluster grande), el tiempo de procesamiento puede crecer proporcionalmente con esa cantidad de reglas, mientras eBPF puede ofrecer un mecanismo de procesamiento más directo y eficiente que no se degrada de la misma forma a medida que el número de reglas aumenta.

Lección 17 — eBPF vs iptables — diferencias de rendimiento

El dataplane eBPF típicamente ofrece menor latencia y mayor throughput de procesamiento de paquetes, especialmente notable cuando el número de reglas de NetworkPolicy es grande, ya que evita el costo de evaluación secuencial que tiene iptables a medida que la cadena de reglas crece.

Con poco tráfico y pocas reglas, el costo de la evaluación secuencial de iptables es mínimo y apenas perceptible en la práctica; a medida que el volumen de tráfico y el número de reglas crecen significativamente en un cluster grande, ese costo de evaluación secuencial se acumula y se vuelve mucho más notable comparado con el procesamiento más eficiente de eBPF, haciendo que la diferencia de rendimiento entre ambos mecanismos sea mucho más perceptible precisamente en esos escenarios de mayor escala.

Lección 18 — DSR (Direct Server Return) con eBPF

Permite que el tráfico de respuesta de un servicio viaje directamente de vuelta hacia el cliente original sin tener que pasar nuevamente a través del nodo que originalmente recibió la petición entrante, reduciendo saltos innecesarios y mejorando la latencia de respuesta en ciertos escenarios de red.

Si el tráfico de respuesta tuviera que volver obligatoriamente a través del nodo que recibió la petición original antes de llegar al cliente, ese salto adicional añadiría tiempo real de procesamiento y tránsito de red; con DSR, el tráfico de respuesta puede ir directamente desde donde se generó esa respuesta hacia el cliente original, evitando ese salto intermedio innecesario y reduciendo la latencia total percibida en el camino de respuesta.

Lección 19 — Habilitar y migrar hacia el dataplane eBPF

Verificar la compatibilidad de la versión del kernel de Linux de los nodos (eBPF requiere una versión de kernel relativamente reciente con soporte adecuado), y planificar la transición de forma controlada para minimizar el riesgo de interrupción, de forma similar a cualquier cambio significativo de infraestructura de red en producción.

eBPF depende de capacidades específicas que solo están disponibles en versiones relativamente recientes del kernel de Linux; si algún nodo del cluster tuviera una versión de kernel más antigua sin ese soporte necesario, intentar habilitar eBPF ahí fallaría o causaría un comportamiento inesperado, razón por

la cual verificar esa compatibilidad de antemano en cada nodo es un paso de planificación necesario antes de proceder con la migración hacia ese nuevo dataplane.

Lección 20 — Limitaciones del dataplane eBPF

Al ser una tecnología relativamente más nueva dentro de Calico comparada con el dataplane tradicional de iptables (mucho más maduro y probado durante más tiempo), eBPF puede tener ciertas limitaciones de compatibilidad con algunas funcionalidades específicas o escenarios menos comunes, que conviene verificar cuidadosamente antes de adoptarlo en un entorno crítico.

Al ser una tecnología relativamente más nueva dentro de Calico, con menos tiempo acumulado de uso real en producción comparado con el dataplane tradicional de iptables, eBPF podría tener casos límite o incompatibilidades con funcionalidades específicas que aún no se han descubierto completamente; probarlo primero en un entorno de prueba permite descubrir esos posibles problemas en un contexto controlado, sin arriesgar directamente la estabilidad de un entorno de producción crítico.

Módulo 5: Seguridad avanzada

Lección 21 — Calico Enterprise/Calico Cloud vs Calico OSS

Capacidades adicionales orientadas a visibilidad y seguridad más avanzada para organizaciones grandes, como dashboards de observabilidad de red más ricos, detección de amenazas, compliance reporting, y soporte comercial oficial, sin estar disponibles en la versión OSS gratuita.

Una organización sujeta a requisitos regulatorios reales necesita poder demostrar y documentar formalmente su postura de seguridad de red ante auditores externos; la capacidad de compliance reporting de Calico Enterprise genera esa documentación necesaria de forma automatizada, mientras la versión OSS carecería de esa herramienta específica, dejando a la organización con la tarea más costosa de construir manualmente esa evidencia de cumplimiento por su cuenta.

Lección 22 — Application layer policy con Istio

Permite definir políticas basadas en atributos de la capa de aplicación (como el método HTTP específico, la ruta de la URL, o la identidad criptográfica del servicio verificada mediante mTLS de Istio), ofreciendo un control mucho más granular que las políticas tradicionales basadas únicamente en IP y puerto.

Una política tradicional de Calico basada en IP y puerto solo tiene visibilidad sobre esa información de capa 3/4, sin ningún conocimiento real del contenido específico de la petición HTTP (como si es un GET o un POST, o hacia qué ruta específica va dirigida); application layer policy añade esa capacidad de inspeccionar y restringir según esos detalles de capa de aplicación más alta, algo que una política tradicional simplemente no tiene forma de distinguir con la información limitada de IP y puerto que maneja.

Lección 23 — Cifrado de datos en tránsito con WireGuard

Permite cifrar automáticamente el tráfico de red entre nodos usando WireGuard (un protocolo de VPN moderno y eficiente), protegiendo los datos en tránsito de ser interceptados en la red física subyacente, sin requerir que las aplicaciones mismas implementen su propio cifrado adicional.

Si ninguna capa implementara cifrado, el tráfico viajaría completamente en texto plano y visible para cualquiera con acceso a la red física subyacente; cifrar a nivel de red con WireGuard protege ese tráfico independientemente de si cada aplicación específica implementó o no su propia capa adicional de cifrado, actuando como una salvaguarda base que cubre el escenario donde la aplicación no tiene cifrado propio, dejando ese tráfico protegido de todas formas gracias a esa capa de red.

Lección 24 — DNS policy — restringir acceso por dominio

Permite definir reglas que permiten o deniegan tráfico hacia un dominio específico (como api.ejemplo.com) sin necesitar conocer de antemano todas las direcciones IP asociadas a ese dominio, que además podrían cambiar con el tiempo, resolviendo dinámicamente esa relación entre dominio y dirección IP real.

Una política basada en direcciones IP fijas especifica exactamente esas direcciones en el momento de su creación; si el proveedor externo cambia sus direcciones IP más adelante (algo común en servicios cloud), esa política fija dejaría de coincidir con las direcciones reales actuales, rompiendo silenciosamente el acceso, mientras una política de DNS resuelve el dominio dinámicamente en cada conexión real, obteniendo siempre la dirección IP actual sin importar si cambió desde que la política fue creada originalmente.

Lección 25 — Federación de clusters con Calico

Permite que Pods o servicios ubicados en un cluster puedan ser referenciados dentro de reglas de NetworkPolicy definidas en otro cluster federado, facilitando mantener políticas de seguridad consistentes en escenarios donde una organización opera múltiples clusters distintos que necesitan comunicarse entre sí.

Sin federación, cada cluster tendría que gestionar la comunicación hacia el otro cluster de forma más genérica, posiblemente basándose solo en rangos de IP sin poder referenciar directamente servicios o Pods específicos del cluster remoto; con la federación de Calico, se pueden definir reglas más precisas que referencian directamente esos recursos reales del otro cluster, manteniendo un control más granular y preciso sobre esa comunicación entre clusters distintos que necesitan coordinarse ocasionalmente.

Módulo 6: Producción avanzada

Lección 26 — Troubleshooting de Calico — calicoctl y diagnóstico

Permite inspeccionar recursos específicos de Calico (como el estado del enrutamiento BGP entre nodos, los perfiles de IPAM, o las GlobalNetworkPolicy) que no son recursos nativos de Kubernetes y por tanto no son directamente visibles ni gestionables únicamente con kubectl.

El estado del enrutamiento BGP es información específica y propia de Calico, que no forma parte de los recursos nativos que kubectl conoce y puede mostrar directamente; kubectl está diseñado para gestionar los recursos estándar de Kubernetes, sin ninguna visibilidad real sobre detalles internos específicos del protocolo BGP, mientras calicoctl sí tiene acceso directo a esa información particular, siendo la herramienta apropiada para diagnosticar ese tipo específico de problema de enrutamiento.

Lección 27 — Monitoreo de Calico con Prometheus

Métricas sobre el número de reglas de NetworkPolicy aplicadas y su tiempo de procesamiento, el estado de las sesiones BGP (activas vs caídas), y el número de paquetes denegados por políticas, dando visibilidad real de si la seguridad y el enrutamiento de red están funcionando correctamente más allá de solo el estado del proceso.

El proceso de Felix podría estar técnicamente en ejecución mientras una sesión específica de BGP con otro nodo está caída, sin que eso se refleje en el estado básico del proceso; esa sesión BGP caída significa que ese nodo específico no está recibiendo o propagando correctamente las rutas necesarias para la conectividad real, un problema que solo una métrica específica del estado de las sesiones BGP revelaría, no la simple confirmación de que el proceso sigue corriendo normalmente.

Lección 28 — Rendimiento y escalabilidad de Calico en clusters grandes

Typha actúa como una capa intermedia de caché y distribución de datos entre los servers de Kubernetes (donde vive el estado) y los agentes Felix de cada nodo, reduciendo la carga directa sobre el API server de Kubernetes que ocurriría si cada uno de miles de Felix consultara directamente ese API server por separado.

Cada uno de los agentes Felix consultaría directamente al mismo API server central compartido por todo el cluster; con miles de esos agentes haciendo consultas simultáneas sin ningún intermediario, la carga acumulada de peticiones podría exceder la capacidad real que ese único API server puede procesar eficientemente, mientras Typha actúa precisamente como una capa intermedia que consolida esas consultas, reduciendo significativamente la carga real que llega hasta el API server central.

Lección 29 — Migración hacia Calico desde otro CNI

Planificar una estrategia que minimice la interrupción del tráfico durante la transición, ya que los Pods existentes seguirían usando el mecanismo de red del CNI anterior hasta ser recreados, considerando probablemente reiniciar los Pods de forma gradual para que adopten Calico sin causar una interrupción completa simultánea del cluster, de forma similar a lo ya visto en el curso de Flannel.

El principio fundamental de que los Pods existentes mantienen su mecanismo de red original hasta ser recreados, y de que reiniciar todos simultáneamente causaría una interrupción completa, aplica sin importar cuáles sean los dos plugins CNI específicos involucrados en la transición; esa consideración de migración gradual es una práctica general aplicable a cualquier cambio de CNI, independiente de los detalles técnicos específicos de cómo cada plugin particular gestiona la red internamente.

Lección 30 — Proyecto final: red de cluster con Calico lista para producción

Modo de encapsulación apropiado elegido según la topología real de red (BGP puro, IP-in-IP o VXLAN según corresponda) + NetworkPolicy bien diseñadas con el modelo deny-by-default + dataplane elegido conscientemente (iptables o eBPF según los requisitos) + MTU correctamente ajustado + monitoreo con Prometheus + procedimiento de diagnóstico con calicoctl documentado.

Intenta mencionar al menos dos pilares: modo de encapsulación apropiado, NetworkPolicy con deny-by-default, elección consciente del dataplane o monitoreo con diagnóstico documentado.